

TABLE 4
STAGE 3-ANALOGUE VOLTAGE SENSE

Analogue Sense 3-Channel	Best Return	Options		
Input	₹170	MCP3204/3208 ADC – SPI ₹365	Arduino Nano ₹450	CD4051 8-channel analogue multiplexer ₹189
Output		SPI	CPU	M.plex
Return		₹365	₹450	₹189

TABLE 5
STAGE 4-CPU OPTIONS

CPU			Options			
Inputs	SPI	₹365		ESP32 ₹745	NODE MCU ₹575	R.Pi Zero W ₹1,715
	CPU	₹450	Arduino Nano ₹450			
	M.Plex	₹189		ESP32 ₹569	Node MCU ₹399	
Outputs			Arduino Nano ₹450	ESP32 ₹569	Node MCU ₹399	R.Pi Zero W ₹1,715

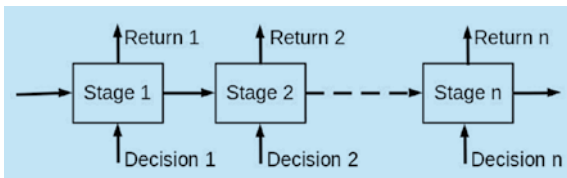


Fig. 1: Dynamic programming problem

At every stage of the design, we have multiple choices. A good product architect must evaluate the impact of his design decisions in totality.

In our earlier article, we discussed concurrent engineering. Making design decisions with a big and diverse team is challenging work. This coupled with the analysing effect of our design in totality create some big challenge.

To understand the problem let us consider we have five modules in our product and our manufacturing has four steps. If we generate even a modest three alternatives for each of these nine steps, we land up with a total of 3^9 or, 19,683 alternate scenarios. Evaluating such a large number of scenarios would be very difficult.

Dynamic programming

There is a branch of mathematics called operations research that

deals with efficient ways to solve business problems. Decision problems that involve finding the best solution requiring a series of decision steps are known as dynamic programming problems.

In dynamic programming we view the entire process as a few distinct steps or stages (see Fig. 1). Each stage receives an input. We control the outcome of each step by changing a decision variable. A combination of input and decision variable produces the output. In addition to output, the process also gives a certain return. The output of each stage becomes the input of the next stage. Our dynamic programming problem is to find out the best return from all the stages to produce the final output.

There are many standard approaches to solving such problems. One of the simplest and most popular methods is the branch and bound algorithm. This method helps us to cleverly reduce the number of evaluations required to find the best solution.

For instance, if we know that out

of our 19,683 combinations, some of the combinations are infeasible, then we can eliminate the entire branch that has the infeasible combination of options. Similarly, if we have reached an option that is costlier than a combination we had found earlier, then we can drop that branch as well.

In the branch and bound method, we find these costlier or infeasible branches and eliminate them from further search. This way we get the best possible combination with fewer number of evaluations.

I prefer to decide on the product architecture manually. There are many powerful computer software for solving these kinds of problems. However, internal workings are not evident when we solve these problems on a computer. Knowledge of the intermediate steps often helps in triggering further innovation.

Inside-out tabular process

The manual process that I use calls for charting the alternatives in a series of tables, I call it an inside-out tabular process. In the table, we put the inputs in rows and the outputs in columns. The options go into the cells. If we have more than one option for input resulting in the same output, we select the best input option for each output. In the next stage, the output of the previous stage becomes input. We evaluate all the stages to find out the best combination of choices.

We use this process to get a first draft version of the charge controller parts. Our first requirement is to divert power from the wind turbine to a dummy load. We can use a solid-state relay or a solenoid relay for this purpose. From the websites of bulk electronic parts suppliers, we can get prices for various options, which are listed in Table 2.

We club options that require a similar output current in the same column. For our design requirements, we have six options and five different current requirements. We can drop